

Glance ML Internship Assignment: Multimodal Fashion & Context Retrieval

Possible Approaches

When building a multimodal search engine for fashion and context, several approaches can be utilized, each with distinct tradeoffs:

Vanilla CLIP (Zero-Shot):

- How it works: Uses pre-trained CLIP to encode both the search query and the images into the same latent space, fetching the highest cosine similarity.
- Pros: Extremely fast, highly scalable, and requires no additional training or complex architecture.
- Cons: As noted in the assignment, it severely struggles with compositionality (attribute binding). It cannot easily distinguish between "red shirt with blue pants" and "blue shirt with red pants".
- When to use: Good as a strong baseline or for simple, single-subject image retrieval where fine-grained details do not matter.

Vision-Language Models (VLMs) (e.g., LLaVA, GPT-4V):

- How it works: Passing the image and text prompt to a large multimodal model to explicitly answer if the image matches the query.
- Pros: Exceptional reasoning and compositional understanding. It solves the attribute binding problem perfectly.
- Cons: Extremely high latency and computationally expensive. It is impossible to use this for fast, real-time vector search across millions of images.
- When to use: Best used in a two-stage retrieval system specifically for re-ranking a small subset (e.g., top 20) of images retrieved by a faster first-stage model.

Fine-Tuned Dual Encoders (Fashion-Specific):

- How it works: Fine-tuning CLIP or another dual-encoder on a fashion-specific dataset (like FashionPedia) using hard negative mining (e.g., feeding images of blue shirts as negative examples for a "red shirt" query).
- Pros: Greatly improves domain-specific precision and attribute binding.
- Cons: Requires substantial compute resources and high-quality, cleanly labeled paired datasets.

- When to use: Ideal for enterprise-grade production systems with access to vast proprietary clickstream or labeled data.

Current Approach (Decomposition + Prompt Ensembling with CLIP):

- How it works: Retains the speed of vanilla CLIP but introduces a semantic text pipeline that splits complex queries into sub-queries, applies domain-specific prompt templates, and aggregates the embeddings.
- Pros: Highly scalable, lightweight, and specifically targets the compositional weaknesses of CLIP without requiring expensive fine-tuning.
- Cons: Still bounded by the underlying vision encoder's zero-shot capabilities.
- When to use: Perfect for this assignment as it relies purely on ML logic, prompt engineering, and mathematical aggregation to improve results efficiently.

Chosen Approach

Architecture Overview The chosen architecture is built on a modular dual-encoder pipeline designed to extract robust representations for both images and text.

- **The Indexer:** The system processes raw images using a vision encoder in batched inferences. The resulting embeddings are L2-normalized to ensure they sit on a unit hypersphere, making cosine similarity computationally efficient via dot products, and then stored in a ChromaDB vector database.
- **The Retriever:** The search logic is designed to accept natural language strings. To handle multi-attribute and compositional queries, the retriever applies a custom ML logic pipeline before vector matching.

Handling Fashion Queries (Compositionality) To overcome vanilla CLIP's weakness with compositionality, the retriever utilizes two custom components:

1. **Decomposer:** When a user submits a complex query (e.g., "A red tie and a white shirt in a formal setting"), the decomposer splits the query into independent semantic segments based on conjunctions. This prevents the text encoder from tangling attributes together.
2. **Encoder with Template Ensembling:** The system analyzes each decomposed segment to detect if it contains context/environment keywords (e.g., office, park) or color keywords. Based on this, it routes the segment through tailored prompt templates. The model embeds all variations of the prompt, L2-normalizes them, and takes the mathematical mean of these embeddings to create a single, highly robust representation of the segment. Finally, multi-segment embeddings are averaged together to query the vector database.

This approach forces the text embedding to focus heavily on the specific fashion attributes and environments requested, significantly boosting zero-shot precision for multi-attribute queries.

Codebase (GitHub) Link

Repository: <https://github.com/dheepakshakthi/Multimodal-Fashion-and-Context-Retrieval>

Approaches for future work

Metadata Filtering (Hybrid Search):

Visual models struggle to accurately guess specific cities (e.g., "New York" vs "Chicago" streets) solely from pixels. To support this, external metadata (GPS coordinates, weather at the time the photo was taken) should be injected during the indexing phase. The vector database (like ChromaDB or Pinecone) can then perform hybrid search: filtering exact metadata (e.g., location="Paris", weather="Rain") and running the semantic vector search on the remaining pool of images.

Visual Weather Augmentation:

For purely visual weather concepts (e.g., "rainy vibe", "snowy"), we can expand the text encoder's template ensembling to include a WEATHER_PROMPT_TEMPLATES list, triggered by weather-related keywords to boost the vector's attention to environmental phenomena.

Two-Stage Re-Ranking (Cross-Encoder/VLM):

Implement a fast Stage-1 retrieval (our current pipeline) to pull the top 50 matches. Then, pass these 50 images alongside the user's prompt to a lighter Vision-Language Model or a specifically trained cross-encoder. The cross-encoder evaluates the image and text pair simultaneously, allowing it to accurately check strict attribute binding (e.g., ensuring the tie is red, not the shirt).

Contrastive Fine-Tuning with Hard Negatives:

Fine-tune the underlying CLIP model by explicitly training it on hard negatives. We can generate synthetic negative captions for our dataset (e.g., swapping colors of garments in the text) and penalize the model if it scores the incorrect textual combination highly. This forces the underlying vision encoder to learn precise localization of fashion attributes.